

Project Plan

The goal of this project is to improve an AI interview coach by using a small open-source language model to generate interview feedback, then using a stronger AI judge to evaluate and guide model improvement.

The base generation model is Qwen2.5-3B-Instruct. It is responsible for simulating student answers, generating baseline coaching feedback, generating preference candidates, and later serving as the trainable model.

The judge model is Qwen3.6-35B-A3B. It is used to score feedback quality and select preferred feedback pairs for RLAIIF training.

Stage 1: Dataset Preparation

Collect and preprocess software engineering interview questions from the available datasets. Each example contains a question, reference answer, category, difficulty level, student answer, and student answer type.

The dataset is split into:

train.jsonl: used for preference candidate generation and training

test.jsonl: held-out evaluation set used for baseline and post-training comparison

Each data record follows this schema:

```
{
  "id": "train_0001",
  "question": "...",
  "reference_answer": "...",
  "category": "...",
  "difficulty": "...",
  "student_answer": "...",
  "student_answer_type": "...",
  "source": "..."
}
```

Stage 2: Baseline Evaluation

Use Qwen2.5-3B-Instruct on the held-out test set to generate coaching feedback for each student answer.

Then use Qwen3.6-35B-A3B to score the baseline feedback based on technical correctness, specificity, helpfulness, actionability, and suitability for interview coaching.

This produces the baseline score before training.

Stage 3: Preference Candidate Generation

Use Qwen2.5-3B-Instruct on the training set to generate two feedback candidates for each question-answer pair.

For each training example:

generate feedback_a

generate feedback_b

use randomized but reproducible feedback styles

record metadata such as style and temperature

The purpose is to create diverse candidate feedback without making one side always better by design.

Stage 4: Preference Pair Selection (RLAIF)

Use Qwen3.6-35B-A3B as an AI judge to compare feedback_a and feedback_b.

The judge selects the better feedback based on substance, not position, length, formatting, or style. The selected feedback becomes the chosen response, and the other becomes the rejected response.

This creates preference pairs for RLAIF training.

Stage 5: SFT Dataset Construction

Construct a supervised fine-tuning dataset from high-quality examples.

The SFT dataset uses the interview question, reference answer, and student answer as input, and uses selected high-quality coaching feedback as the target output.

This prepares the model to learn the general format and behavior of interview coaching before preference optimization.

Stage 6: SFT with QLoRA

Fine-tune Qwen2.5-3B-Instruct using supervised fine-tuning with QLoRA.

QLoRA is used to reduce GPU memory cost by training lightweight adapter weights instead of fully fine-tuning all model parameters.

The SFT model becomes the intermediate improved model.

Stage 7: DPO Training with QLoRA (RLAIF)

Use the preference pairs selected by Qwen3.6-35B-A3B to train the model with Direct Preference Optimization.

The model learns to prefer feedback that is more technically correct, specific, helpful, actionable, and suitable for interview coaching.

QLoRA is again used to make training feasible with limited compute.

Stage 8: New Model Evaluation

Use the trained model on the same held-out test.jsonl set used in the baseline stage.

For each test example, generate new coaching feedback using the trained model.

Then use Qwen3.6-35B-A3B to score the new feedback using the same rubric as the baseline evaluation.

Stage 9: Baseline vs New Model Comparison

Compare baseline scores and new model scores on the same 200 test examples.

The comparison measures whether training improved feedback quality.

Evaluation metrics include:

average score improvement

technical correctness

specificity

helpfulness

actionability

interview coaching suitability

win rate between baseline and trained model feedback

Stage 10: Small-scale Human Evaluation

Conduct a small human evaluation to validate whether the AI judge scores are reasonable.

Human evaluators compare a subset of baseline and trained model feedback. They judge which feedback is more useful, accurate, and actionable for a student preparing for software engineering interviews.

This provides additional evidence beyond automated scoring.

Detail

Detailed Experimental Plan

1. Dataset Source and Size

The dataset is built from two software engineering interview question sources: Software Questions and ai_interview_questions.

After preprocessing, the data is split into:

Training set: approximately 4,653 examples

Test set: 200 held-out examples

Each example contains an interview question, a reference answer, metadata such as category and difficulty, and a simulated student answer.

The test set is kept fixed throughout the experiment. The same test questions and the same student answers are used for both baseline evaluation and final model evaluation. Student answers are not regenerated during the comparison stage.

2. Baseline Evaluation Setup

The baseline model is Qwen2.5-3B-Instruct.

For each example in the fixed test set, the model generates coaching feedback based on:

- the interview question
- the reference answer
- the fixed student answer
- The output is saved as baseline feedback.

The baseline feedback is then scored by the judge model, Qwen3.6-35B-A3B, using the same evaluation rubric that will later be used for the trained model.

This creates the baseline performance score before fine-tuning.

3. Student Answer Simulation Setup

Student answers are generated before training and evaluation. They are designed to simulate a first-year CS master’s student answering software engineering interview questions.

The simulated answers include different answer quality types, such as:

- correct but incomplete
- partially correct
- incorrect
- too vague
- verbose but unfocused

Once generated, these student answers are fixed. They are not regenerated for baseline evaluation, preference candidate generation, or final model evaluation. This ensures that any score difference comes from changes in the coaching feedback model, not from changes in student answers.

4. Preference Candidate Generation and Bias Control

For the training set, Qwen2.5-3B-Instruct generates two coaching feedback candidates for each fixed student answer:

feedback_a

feedback_b

To avoid making one side consistently better by design, the system uses randomized but reproducible feedback styles. The A/B assignment is not tied to a fixed quality level, and the model records metadata such as:

- feedback style
- generation temperature
- candidate model name

This reduces the risk that the preference model learns superficial patterns such as always preferring feedback_b, longer answers, or one specific feedback style.

5. Judge Rubric and Preference Selection

The judge model is Qwen3.6-35B-A3B.

For baseline scoring, it evaluates each feedback response using a rubric based on:

- technical correctness
- specificity
- helpfulness
- actionability
- suitability for software engineering interview coaching

For preference selection, the judge compares feedback_a and feedback_b and selects the better one. The prompt explicitly instructs the judge not to prefer an answer because of:

- A/B position
- length
- formatting
- tone alone
- style label

The selected response becomes the chosen feedback, and the other response becomes the rejected feedback for later RLAIIF / DPO training.